Zach Lowe, EECS 348

Gen AI Code Comparison – Assignment 5

In this assignment, I chose to compare the way that the free browser models available for both Copilot and Claude fulfilled the following prompt while using the puzzle files and assignment 5 rubric provided on canvas as guidelines for them to follow in revising another student's assignment 4, alongside the student's provided UML Use Case Diagram, and the prompt I used for assignment 4 as another reference to the expected function and required criteria:

I am giving you 5 puzzle files, a C++ file titled "Gemini_revised" that I will refer to as assignment 4 code, a rubric and instruction file for Assignment 5 to use as an additional frame of reference, and a UML use case diagram provided by the student who made the code. Modify the Assignment 4 code you receive as follows for the purpose of assignment 5: Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number. Each solutionN.txt file should include the following: If no solutions are found, a single line: “No solution found.” If solutions are found, it should include all solutions followed by a single line: “Solutions found: n”, where n is the number of solutions found. The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt. Do not replace the entire file with your code. You must get all required functionality from Assignment 4 working, even if the student you inherited it from did not. The requirements for assignment 4 were as follows:

Generate C++ code using objects, not functions, to implement the following requirements: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that. It is possible to solve a Sudoku puzzle entirely through logic. We’re going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier). You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square. Write a recursive object-oriented program to find the solution to the five puzzle files. The output for each solution, should include: The puzzle file name (e.g., puzzle1.txt). The puzzle from the file printed out. The solution to the puzzle printed out or if no solution is found, “No solution found” printed out. The solution may not be unique; if

there is more than one solution to the puzzle, your program should print them all out. To help you debug your program, solution1.txt is a solution to puzzle1.txt.

```
//Revised by: [YOUR FULL NAME]   Revision date: 10/29/2026
//Assignment 5 revision: solutions are now also written to solutionN.txt (N = puzzle number).
//  Each file holds every solution in the same format as puzzleN.txt (9 rows, '_' = blank)
//  followed by "Solutions found: n", or just "No solution found." if none exist.
//  Inputs: puzzle1.txt - puzzle5.txt   Outputs: console text plus solution1.txt - solution5.txt
//  Revision collaborator: Claude (Anthropic) helped write the revision code marked "A5" below.
//platform Gemini website engine
//Yunjingma Assigment04
//author yunjing ma



//Assignment4.cpp
// Sudoku Solver using Object-Oriented Programming and Recursive Backtracking.
//EECS 348 Software Engineering I - Assignment 4

// Description:
// This program reads five Sudoku puzzle files (puzzle1.txt through puzzle5.txt),
// validates the input data, and solves each puzzle using a recursive depth-first
// search algorithm with backtracking. It prints the input filename, the original
// puzzle, and all valid solutions found (or a message if no solution exists).
//

#include <iostream>
#include <fstream>
```

```
#include <string>
#include <vector>



//class SudokuSolver
// @brief Encapsulates a 9x9 Sudoku board, file loading logic, validation checks,
// and the recursive backtracking solving algorithm.
class SudokuSolver {
private:
  int board[9][9];      // 9x9 grid representing the Sudoku puzzle (0 denotes empty cells)
  std::string filename;  // The input file name associated with this puzzle instance
  std::string solutionFilename; // A5: output file name, e.g. "solution1.txt" (built from the puzzle name)
  std::ofstream solutionOut;    // A5: stream used to write every solution found to solutionFilename


  // A5: writes the current board to the solution file in the same layout as the puzzle input files
  void writeBoardToFile() {
    for (int r = 0; r < 9; ++r) {                 // go through each of the 9 rows
      for (int c = 0; c < 9; ++c) {               // go through each of the 9 columns
        solutionOut << (board[r][c] == 0 ? '_' : char('0' + board[r][c])); // digit, or '_' if blank
        if (c < 8) solutionOut << ' ';            // single space between cells, none after the last
      }
      solutionOut << "\n";                        // end the row
    }
  }

// brief Checks if placing a number at a specific cell violates Sudoku rules.
    // Check row and column constraints
```

```
  bool isValid(int row, int col, int num) const {
    for (int i = 0; i < 9; ++i) {
      if (board[row][i] == num) return false;//row Current row index (0-8)if any number repeat twice
      if (board[i][col] == num) return false;//colun Current row index (0-8) any number repeat twice

    }

    // Check 3x3 subgrid constraints
    int startRow = row - (row % 3);// it will out make until one loop neds all will start at  first column thrid and sixth three times to check each 3*3 grid of the row
    int startCol = col - (col % 3);
    for (int r = 0; r < 3; ++r) {
      for (int c = 0; c < 3; ++c) {
        if (board[startRow + r][startCol + c] == num) {
          return false;//if any of 3*3 a repeat number return flase
        }
      }
    }

    return true;
  }
  bool validateInitialBoard() const {
// the 9x9 Sudoku board.
    for (int r = 0; r < 9; ++r) {
// Visit every column in row
      for (int c = 0; c < 9; ++c) {
// Store the value of the spot
        int num = board[r][c];
```

```
// blank as 0.
            if (num == 0) {
               continue;
            }
// Check the current row and column for duplicates in the row and column.
            for (int k = 0; k < 9; ++k) {
// Check the current row. k != c abiod compare the cell with itself.
               if (k != c && board[r][k] == num) {
                  return false;
               }
// Check the current column. if k != r prevents comparing the cell with itself.
               if (k != r && board[k][c] == num) {
                  return false;
               }
            }
// Integer division groups rows 0-2, 3-5, and 6-8.
            int startRow = (r / 3) * 3;
// Calculate the top-left column of the same 3*3grid. Integer division groups columns 0-2,
3-5, and 6-8.
            int startCol = (c / 3) * 3;
            // check all three rows of the current 3x3grid.
            for (int i = startRow; i < startRow + 3; ++i) {
//check all three columns of the subgrid.
               for (int j = startCol; j < startCol + 3; ++j) {
// Check if there's any repition error
                  if ((i != r || j != c) &&
                     board[i][j] == num) {
// A duplicate means the initial puzzle violates Sudoku rules.
                     return false;
```

```
            }
          }
        }
      }
    }

    // Every given number passed all three checks.
    return true;
  }
// @brief Recursive helper function implementing DFS with backtracking to find all solutions.

  void solveRecursive(int row, int col, int &solutionCount) {
// Base case: If row reaches 9, we have successfully filled the entire grid
    if (row == 9) {// pass without the error so it count as one solution
      solutionCount++;//One solution count
      std::cout << "Solution " << solutionCount << ":\n";
      printBoard();// output ofthe result string
      if (solutionCount > 1) solutionOut << "\n"; // A5: blank line separates one solution from the previous one
      writeBoardToFile();                 // A5: save this solution to solutionN.txt
      return;
    }

// Determine coordinates for the next cell
    int nextRow = (col == 8) ? row + 1 : row;//row check done swtich to next row
    int nextCol = (col == 8) ? 0 : col + 1;// if column done return zero to next row index 0
// If the current cell is already pre-filled, skip to the next cell
    if (board[row][col] != 0) {// if not blank in the soduku
```

```
        solveRecursive(nextRow, nextCol, solutionCount);
        return;
      }

// Try placing digits 1 through 9 in the empty cell
      for (int num = 1; num <= 9; ++num) {
        if (isValid(row, col, num)) {// trying the number 1 to 9
          board[row][col] = num;

// Recurse to the next cell
          solveRecursive(nextRow, nextCol, solutionCount);

          board[row][col] = 0;   // Backtrack if it fail and try another branch
        }
      }
    }

public:
// @brief Constructor that initializes the Sudoku solver with a target filename.
// @param fname The name of the input text file.

   SudokuSolver(const std::string &fname) : filename(fname) {//initialize and the read the
file one by one
      // A5: "puzzleN.txt" -> "solutionN.txt" by swapping the "puzzle" prefix for "solution"
      size_t pos = fname.find("puzzle");
      solutionFilename = (pos == std::string::npos) ? "solution_" + fname
                                    : fname.substr(0, pos) + "solution" + fname.substr(pos + 6);
// Initialize board with zeros as preparing for the read file
      for (int i = 0; i < 9; ++i) {
```

```
        for (int j = 0; j < 9; ++j) {
            board[i][j] = 0;
        }
    }
  }

// @brief Loads and parses the Sudoku puzzle from the input file.
// return true if successfully loaded and validated, false on error.

bool loadPuzzle() {
    std::ifstream file(filename);
    if (!file.is_open()) {//raise an error if the file can't be read
        std::cerr << "Error: Could not open file " << filename << "\n";
        return false;
    }

    int row = 0;
    std::string line;
    while (getline(file, line)) {
        // Ignore empty lines
        // A5 fix: also skip lines holding only spaces/tabs/'\r' (puzzle2.txt ends with such
lines,
        // which Assignment 4 wrongly counted as extra rows)
        if (line.find_first_not_of(" \t\r\n") == std::string::npos) continue;

        if (row >= 9) {//if row over 9 riase an error in the 9*9 sudoku board
            std::cerr << "Error: File " << filename << " contains more than 9 rows.\n";
            return false;
        }
```

```
        int col = 0;// start from the first row
        for (char ch : line) {
            // Skip whitespace characters
            if (ch == ' ' || ch == '\t' || ch == '\r' || ch == '\n') continue;


            if (col >= 9) {//raise an error is col greater than 9 in an 9*9 sudoku board
                std::cerr << "Error: Row " << row + 1 << " in " << filename << " exceeds 9 columns.\n";
                return false;
            }


            // Parse cell value: '_' or '0' or '.' represents blank cell
            if (ch == '_' || ch == '0' || ch == '.') {
                board[row][col] = 0;
            } else if (ch >= '1' && ch <= '9') {//load the number from toe orignal sudoku board
                board[row][col] = ch - '0';
            } else {//riase the error if aother number exist
                std::cerr << "Error: Invalid character '" << ch << "' in " << filename << ".\n";
                return false;
            }
            col++;//swtich to the next col
        }
        if (col != 9) {//raies an errow if the column not 9
            std::cerr << "Error: Row " << row + 1 << " in " << filename << " has " << col << " columns (expected 9).\n";
            return false;
        }
        row++;// swtich to the next row
```

```
    }
    file.close();// close the file
    if (row != 9) {//raise the error if the row is not 9
      std::cerr << "Error: File " << filename << " contains only " << row << " rows (expected
9).\n";
      return false;
    }

    return true;
  }
//else return true

//    @brief Prints the current state of the Sudoku board in a formatted grid.
  void printBoard() const {// print the board line
    for (int r = 0; r < 9; ++r) {
      if (r > 0 && r % 3 == 0) {// print the horrizonal board line if this row can be devide by 3
        std::cout << "------+-------+------\n";
      }
      for (int c = 0; c < 9; ++c) {// print the column line at 3 6 9
        if (c > 0 && c % 3 == 0) {
          std::cout << "| ";
        }
        if (board[r][c] == 0) {//print the outside of the sukodo boarder outine
          std::cout << "_ ";
        } else {
          std::cout << board[r][c] << " ";//leave a blank for other
        }
      }
      std::cout << "\n";
```

```
    }
    std::cout << "\n";
  }


// @brief Executes the solver on the loaded puzzle and prints results.
  void solve() {
    std::cout << "========================================\n";
    std::cout << "Input Filename: " << filename << "\n";
    std::cout << "Original Puzzle:\n";
    printBoard();// print the board

    solutionOut.open(solutionFilename);            // A5: create/overwrite solutionN.txt
    if (!solutionOut.is_open()) {                  // A5: report if the output file cannot be created
      std::cerr << "Error: Could not open file " << solutionFilename << " for writing.\n";
    }

    int solutionCount = 0;//the solution virable
    // A5: Assignment 4 never called validateInitialBoard(); a puzzle whose givens already
    // conflict has no solution, so we skip the search instead of solving an invalid board.
    if (validateInitialBoard()) {
      solveRecursive(0, 0, solutionCount);//no solution virable
    }

    if (solutionCount == 0) {//if solution not found the output string
      std::cout << "No solution found.\n";
      solutionOut << "No solution found.\n";                  // A5: file holds only this line
    } else {//other condition output string
```

```
        std::cout << "Total solutions found: " << solutionCount << "\n";
        solutionOut << "Solutions found: " << solutionCount << "\n"; // A5: required final line
    }
    solutionOut.close();                    // A5: flush and close the solution file
    std::cout << "========================================\n\n";
  }
};

/**
 * @brief Main execution function that processes puzzle1.txt through puzzle5.txt.
 */
int main() {
  // Array containing the names of the five required puzzle files
  std::string puzzleFiles[5] = {// the name of the five file
    "puzzle1.txt",
    "puzzle2.txt",
    "puzzle3.txt",
    "puzzle4.txt",
    "puzzle5.txt"
  };

  // Iterate through each puzzle file and solve it
  for (int i = 0; i < 5; ++i) {// solve the 5 puzzles one by one suing for loops
    SudokuSolver solver(puzzleFiles[i]);

    // Load puzzle and check for reading/format errors before solving
    if (solver.loadPuzzle()) {// if it can be read print the solution
      solver.solve();
```

```
    } else {//raise an error if the puzzle  file can't be read
        std::cout << "Skipping " << puzzleFiles[i] << " due to input errors.\n\n";
    }
  }

  return 0;
}
```

Figure 1 (Claude Initial Code)

```
//platform Gemini website engine
//Yunjingma Assigment04
//author yunjing ma
//Assignment4.cpp
// Sudoku Solver using Object-Oriented Programming and Recursive Backtracking.
// EECS 348 Software Engineering I - Assignment 4
//
// Revised by: Zach Lowe
// Assignment 5 Modifications
// Revision Date: 10/29/2026
//
// Description:
// This program reads five Sudoku puzzle files (puzzle1.txt through puzzle5.txt),
// validates the input data, and solves each puzzle using a recursive depth-first
// search algorithm with backtracking. It prints the input filename, the original
// puzzle, and all valid solutions found (or a message if no solution exists).
//

#include <iostream>
```

```
#include <fstream>
#include <string>
#include <vector>
#include <sstream>

//class SudokuSolver
// @brief Encapsulates a 9x9 Sudoku board, file loading logic, validation checks,
// and the recursive backtracking solving algorithm.
class SudokuSolver {
private:
   int board[9][9];
   std::string filename;
   std::vector<std::string> solutions;

   bool isValid(int row, int col, int num) const {
      for (int i = 0; i < 9; ++i) {
         if (board[row][i] == num) return false;
         if (board[i][col] == num) return false;
      }

      int startRow = row - (row % 3);
      int startCol = col - (col % 3);

      for (int r = 0; r < 3; ++r) {
         for (int c = 0; c < 3; ++c) {
            if (board[startRow + r][startCol + c] == num) {
               return false;
            }
```

```
      }
    }

    return true;
}

bool validateInitialBoard() const {
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            int num = board[r][c];

            if (num == 0) {
                continue;
            }

            for (int k = 0; k < 9; ++k) {
                if (k != c && board[r][k] == num) {
                    return false;
                }

                if (k != r && board[k][c] == num) {
                    return false;
                }
            }

            int startRow = (r / 3) * 3;
            int startCol = (c / 3) * 3;
```

```
        for (int i = startRow; i < startRow + 3; ++i) {
          for (int j = startCol; j < startCol + 3; ++j) {
            if ((i != r || j != c) &&
              board[i][j] == num) {
              return false;
            }
          }
        }
      }
    }

    return true;
}

std::string boardToString() const {
    std::stringstream ss;

    for (int r = 0; r < 9; ++r) {
      for (int c = 0; c < 9; ++c) {
        ss << board[r][c];
      }
      ss << '\n';
    }

    return ss.str();
}

void saveSolutionsToFile() const {
```

```
std::string solutionFile = filename;
solutionFile.replace(0, 6, "solution");

std::ofstream out(solutionFile);

if (!out.is_open()) {
   std::cerr << "Error creating " << solutionFile << "\n";
   return;
}

if (solutions.empty()) {
   out << "No solution found.\n";
}
else {
   for (size_t i = 0; i < solutions.size(); ++i) {
      out << solutions[i];

      if (i + 1 < solutions.size()) {
         out << '\n';
      }
   }

   out << "Solutions found: "
      << solutions.size()
      << "\n";
}

out.close();
```

```
}

void solveRecursive(int row, int col, int &solutionCount) {
  if (row == 9) {
    solutionCount++;

    solutions.push_back(boardToString());

    std::cout << "Solution "
          << solutionCount
          << ":\n";

    printBoard();
    return;
  }

  int nextRow = (col == 8) ? row + 1 : row;
  int nextCol = (col == 8) ? 0 : col + 1;

  if (board[row][col] != 0) {
    solveRecursive(nextRow, nextCol, solutionCount);
    return;
  }

  for (int num = 1; num <= 9; ++num) {
    if (isValid(row, col, num)) {
      board[row][col] = num;
      solveRecursive(nextRow, nextCol, solutionCount);
```

```
            board[row][col] = 0;
        }
      }
    }

public:
    SudokuSolver(const std::string &fname) : filename(fname) {
      for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
          board[i][j] = 0;
        }
      }
    }

    bool loadPuzzle() {
      std::ifstream file(filename);

      if (!file.is_open()) {
        std::cerr << "Error: Could not open file "
              << filename << "\n";
        return false;
      }

      int row = 0;
      std::string line;

      while (getline(file, line)) {
        if (line.empty()) continue;
```

```
if (row >= 9) {
   std::cerr << "Error: File "
         << filename
         << " contains more than 9 rows.\n";
   return false;
}

int col = 0;

for (char ch : line) {
   if (ch == ' ' || ch == '\t' ||
      ch == '\r' || ch == '\n') {
      continue;
   }

   if (col >= 9) {
      std::cerr << "Error: Row "
            << row + 1
            << " in "
            << filename
            << " exceeds 9 columns.\n";
      return false;
   }

   if (ch == '_' || ch == '0' || ch == '.') {
      board[row][col] = 0;
   }
```

```
      else if (ch >= '1' && ch <= '9') {
        board[row][col] = ch - '0';
      }
      else {
        std::cerr << "Error: Invalid character '"
                  << ch
                  << "' in "
                  << filename
                  << ".\n";
        return false;
      }

      col++;
    }

    if (col != 9) {
      std::cerr << "Error: Row "
                << row + 1
                << " in "
                << filename
                << " has "
                << col
                << " columns (expected 9).\n";
      return false;
    }

    row++;
  }
```

```
    file.close();

    if (row != 9) {
        std::cerr << "Error: File "
                  << filename
                  << " contains only "
                  << row
                  << " rows (expected 9).\n";
        return false;
    }

    if (!validateInitialBoard()) {
        std::cerr << "Error: Invalid Sudoku puzzle in "
                  << filename
                  << "\n";
        return false;
    }

    return true;
}

void printBoard() const {
    for (int r = 0; r < 9; ++r) {
        if (r > 0 && r % 3 == 0) {
            std::cout << "------+-------+------\n";
        }
```

```
      for (int c = 0; c < 9; ++c) {
        if (c > 0 && c % 3 == 0) {
          std::cout << "| ";
        }

        if (board[r][c] == 0) {
          std::cout << "_ ";
        }
        else {
          std::cout << board[r][c] << " ";
        }
      }

      std::cout << "\n";
    }

    std::cout << "\n";
}

void solve() {
    std::cout << "=======================================\n";
    std::cout << "Input Filename: " << filename << "\n";
    std::cout << "Original Puzzle:\n";

    printBoard();

    int solutionCount = 0;
```

```
        solveRecursive(0, 0, solutionCount);

        saveSolutionsToFile();

        if (solutionCount == 0) {
            std::cout << "No solution found.\n";
        }
        else {
            std::cout << "Total solutions found: "
                      << solutionCount
                      << "\n";
        }

        std::cout << "========================================\n\n";
    }
};

/**
 * @brief Main execution function that processes puzzle1.txt through puzzle5.txt.
 */
int main() {
    std::string puzzleFiles[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };
```

```
  for (int i = 0; i < 5; ++i) {
    SudokuSolver solver(puzzleFiles[i]);

    if (solver.loadPuzzle()) {
      solver.solve();
    }
    else {
      std::cout << "Skipping "
            << puzzleFiles[i]
            << " due to input errors.\n\n";
    }
  }

  return 0;
}
```

Figure 2 (Copilot Initial Code)

In terms of correctness, Claude's code performs better than Copilot's, because while both write solutionN.txt correctly for solvable puzzles, Claude's version also fixes an Assignment 4 bug by skipping lines that contain only spaces, tabs or \r. Copilot's version still counts those lines as extra rows and rejects the puzzle. Claude also handles a puzzle with conflicting starting clues by writing "No solution found." to both the console and its solution file. Copilot's version skips that puzzle and writes no file, which leaves the output incomplete and can leave an old file from a previous run. In terms of execution time, the two are effectively equal. Neither changed the recursive search or isValid, so both are O(9^k) in the worst case for k blank cells. Each adds one pass of starting-board validation, which is trivial next to the search. In terms of space complexity, Claude has a clear advantage in that it writes each solution to the file as soon as it is found, so its extra memory stays constant no matter how many solutions exist. Copilot stores every solution in a vector<string> and writes them all at the end, which grows as O(S × 81) for S solutions. In terms of maintainability, while Copilot technically has the cleaner structure with

separate boardToString() and saveSolutionsToFile() methods. However, it removed nearly all of the original comments and reformatted the whole file, which goes against the instruction not to replace the entire file, and circles back around to being a failure in correctness. Claude kept the Assignment 4 code intact and marked each change with an A5 comment explaining why. Given Claude's advantage in correctness, extra memory, and more traceable changes, I chose to continue with Claude's code.

With the revisions, my focus was on improving correctness and maintainability. The existing execution time and space complexity were already sufficient for the purpose of this assignment, so I left the search algorithm and memory usage unchanged and prioritized improvements that would make the solver more reliable and easier to maintain. To improve correctness, I added a final check confirming that each completed board is a valid solution before it is counted, printed, or written to the solution file. To improve maintainability, I replaced the separate starting-puzzle validation routine with a single rule-checking method that can validate both the initial clues and the completed board, so the row, column, and sub-grid rules are written in only one place. I also replaced the repeated values 9, 3, and 0 with named constants, and I cleaned up the header and comments by restoring the original attribution, fixing spelling errors, correcting an inaccurate method description, and removing an unused include.